

Balloon Pop Project File Structure

<https://gemini.google.com/app/090381fa137d522b>

Response: The text you provided appears to be the raw contents or a text dump of an archive file (such as a ZIP or TAR) for a Python game project called "balloon-pop".

Here is the directory and file structure extracted from the data:

- **Root Directory:** 05_balloon_pop-main/
- **Configuration & Documentation Files:** .gitignore, README.md
- **Main Application Directory:** 05_balloon_pop-main/balloon-pop/
 - main.py
 - requirements.txt (This specifies a dependency of pygame>=2.5.0)
- **Game Module Directory:** 05_balloon_pop-main/balloon-pop/game/
 - __init__.py
 - balloon.py
 - click_detection.py
 - game_engine.py
 - renderer.py

How would you like to proceed with this project? I can help you write the code for any of these modules if you are building this game.

*User prompt: Please fix the click-detection bug in my Python game based on the following instructions: The Issue: The check_pop function currently computes the squared distance from the click to the balloon's center, but compares it directly against the balloon's non-squared radius. Because of this, clicks near the edge of the balloon fail to register. I need to fix this so it compares the actual linear distance to the balloon's actual radius. Current Code: Python def check_pop(balloons, click_pos): """ Returns the balloon that was clicked, or None if the click missed every balloon. """ for balloon in balloons: dx = click_pos[0] - balloon.x dy = click_pos[1] - balloon.y distance_squared = dx * dx + dy * dy if distance_squared <= balloon.radius: return balloon return None*

Response:

```
def check_pop(balloons, click_pos):
    """
    Returns the balloon that was clicked, or None if the click missed
    every balloon.
    """
    for balloon in balloons:
        dx = click_pos[0] - balloon.x
        dy = click_pos[1] - balloon.y
        # Calculate the actual linear distance by taking the square root
        distance = (dx * dx + dy * dy) ** 0.5

        if distance <= balloon.radius:
            return balloon
    return None
```

By taking the square root of your `dx * dx + dy * dy` calculation (using `** 0.5`), you convert the squared distance into the actual linear distance. This allows you to accurately compare it against the balloon's actual radius without needing to import any additional math modules.

Alternatively, if you prefer to avoid square root calculations for performance reasons, you can keep your original `distance_squared` calculation and instead compare it to `balloon.radius ** 2`.

User prompt: Please help me implement Task 2 for my Python balloon-popping game. Here are the requirements for the task: The Goal: Introduce at least three different balloon types: Normal Balloon: Grants regular points. Bonus Balloon: Grants more points. Penalty Balloon: Reduces the score (or applies a negative effect). Requirements: Each balloon type must have a distinct color so they are easy to tell apart. The correct score effect must be applied when each specific type is popped. What I need: The updated Python code for the Balloon class (likely in game/balloon.py) to support these different types, colors, and point values. The code for how to randomly spawn these different types of balloons. The code for how to update the score correctly when a specific balloon is popped (likely in game/game_engine.py). Please provide the necessary code snippets and tell me exactly which files to put them in.

Response: To implement the different balloon types, you will need to update the Balloon class to hold properties for type, color, and points, and then update your game loop/engine to handle the spawning and scoring.

1. The Balloon Class

File: game/balloon.py

Update your Balloon class to assign colors and point values based on the balloon type.

```
import random

# Define the properties for each balloon type (Color as RGB tuples)
BALLOON_TYPES = {
    "Normal": {"color": (255, 0, 0), "points": 10},      # Red
    "Bonus": {"color": (255, 215, 0), "points": 50},    # Gold
    "Penalty": {"color": (0, 0, 0), "points": -20}      # Black
}

class Balloon:
    def __init__(self, x, y, radius=30, balloon_type=None):
        self.x = x
        self.y = y
        self.radius = radius

        # If no specific type is passed, randomly select one
        # Weights ensure Normal balloons spawn more often than Bonus or Penalty
        if balloon_type is None:
            self.balloon_type = random.choices(
                ["Normal", "Bonus", "Penalty"],
                weights=[70, 15, 15],
                k=1
            )[0]
        else:
            self.balloon_type = balloon_type

        # Apply the correct color and point value based on the selected type
        properties = BALLOON_TYPES[self.balloon_type]
        self.color = properties["color"]
        self.points = properties["points"]

        # Basic upward movement speed (adjust as needed)
        self.speed = random.randint(1, 3)

    def move(self):
        """Moves the balloon upward."""
        self.y -= self.speed
```

2. Spawning and Scoring Logic

File: game/game_engine.py (or wherever your main game loop/state is managed)

You need functions to generate the balloons at random positions and to process the score when a player successfully clicks one.

```
import random
from game.balloon import Balloon
from game.click_detection import check_pop

class GameEngine:
    def __init__(self):
        self.score = 0
        self.balloons = []

    def spawn_balloon(self, screen_width, screen_height):
        """
        Creates a new balloon at a random horizontal position just below the screen.
        The balloon type is automatically randomized by the Balloon __init__ function.
        """
        x_pos = random.randint(30, screen_width - 30)
        y_pos = screen_height + 30

        new_balloon = Balloon(x_pos, y_pos)
        self.balloons.append(new_balloon)

    def process_click(self, click_pos):
        """
        Checks if a balloon was clicked, updates the score based on the balloon type,
        and removes the balloon from the screen.
        """
        popped_balloon = check_pop(self.balloons, click_pos)
```

```

if popped_balloon:
    # Add or subtract points based on the balloon's specific point value
    self.score += popped_balloon.points

    # Remove the popped balloon from the active list
    self.balloons.remove(popped_balloon)

    # Optional: Prevent score from dropping below 0
    if self.score < 0:
        self.score = 0

    return True # Indicates a successful pop

return False # Indicates a miss

```

User prompt: Here is a prompt you can copy and paste to get the code for Task 3. Please help me implement Task 3 for my Python balloon-popping game. Here are the requirements: The Goal: Implement a lives and miss system. Requirements: The player starts with 3 lives. Whenever a balloon reaches the bottom of the screen without being popped, the player loses 1 life. The remaining lives must be displayed on the screen. End the game once lives reach zero (e.g., stop spawning/moving balloons and display a "Game Over" message). Popping a balloon should never cost a life, regardless of the balloon type. Here is my current game/game_engine.py code. Please update it to include the new lives system and game over logic: """ GameEngine: owns all balloons, spawns new ones, and handles clicks. """ import random from game.balloon import Balloon from game.click_detection import check_pop from game.renderer import WIDTH, HEIGHT SPAWN_INTERVAL_FRAMES = 45 class GameEngine: def __init__(self): self.balloons = [] self.frames_until_spawn = 0 self.score = 0 def _spawn_balloon(self): radius = random.randint(16, 44) x = random.randint(radius + 10, WIDTH - radius - 10) speed = random.uniform(1.5, 3.0) # The Balloon class will now automatically assign a type, color, and point value self.balloons.append(Balloon(x=x, y=-radius, radius=radius, speed=speed)) def handle_click(self, pos): popped = check_pop(self.balloons, pos) if popped is not None: self.balloons.remove(popped) # Add or subtract points based on the specific balloon popped self.score += popped.points # Prevent the score from dropping below 0 if self.score < 0: self.score = 0 def update(self): self.frames_until_spawn -= 1 if self.frames_until_spawn <= 0: self._spawn_balloon() self.frames_until_spawn = SPAWN_INTERVAL_FRAMES for b in self.balloons: b.update() self.balloons = [b for b in self.balloons if not b.is_past_bottom(HEIGHT)] def draw(self, surface, font): from game import renderer renderer.draw_scene(surface, self.balloons) renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))

Response: Replace the contents of game/game_engine.py with the following code. This incorporates the self.lives and self.game_over state variables, handles life deductions when balloons drop past the bottom of the screen, and prevents updates once lives reach zero.

```

"""
GameEngine: owns all balloons, spawns new ones, and handles clicks.
"""

import random

from game.balloon import Balloon
from game.click_detection import check_pop
from game.renderer import WIDTH, HEIGHT

SPAWN_INTERVAL_FRAMES = 45

class GameEngine:
    def __init__(self):
        self.balloons = []
        self.frames_until_spawn = 0
        self.score = 0
        self.lives = 3
        self.game_over = False

    def _spawn_balloon(self):
        radius = random.randint(16, 44)
        x = random.randint(radius + 10, WIDTH - radius - 10)
        speed = random.uniform(1.5, 3.0)
        # The Balloon class will now automatically assign a type, color, and point value
        self.balloons.append(Balloon(x=x, y=-radius, radius=radius, speed=speed))

    def handle_click(self, pos):
        # Ignore clicks if the game is over
        if self.game_over:
            return

        popped = check_pop(self.balloons, pos)
        if popped is not None:
            self.balloons.remove(popped)
            # Add or subtract points based on the specific balloon popped
            self.score += popped.points

```

```

        # Prevent the score from dropping below 0
        if self.score < 0:
            self.score = 0

    def update(self):
        # Stop spawning and moving balloons if the game is over
        if self.game_over:
            return

        self.frames_until_spawn -= 1
        if self.frames_until_spawn <= 0:
            self._spawn_balloon()
            self.frames_until_spawn = SPAWN_INTERVAL_FRAMES

        for b in self.balloons:
            b.update()

        # Rebuild the balloon list, deducting a life for any that drop off-screen
        active_balloons = []
        for b in self.balloons:
            if b.is_past_bottom(HEIGHT):
                self.lives -= 1
            else:
                active_balloons.append(b)

        self.balloons = active_balloons

        # Trigger game over state if lives run out
        if self.lives <= 0:
            self.lives = 0
            self.game_over = True

    def draw(self, surface, font):
        from game import renderer
        renderer.draw_scene(surface, self.balloons)

        # Display Score and Lives
        renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))
        renderer.draw_text(surface, font, f"Lives: {self.lives}", (10, 40))

        # Display Game Over message
        if self.game_over:
            # Basic centering logic; adjust coordinates based on your font size
            renderer.draw_text(surface, font, "GAME OVER", (WIDTH // 2 - 60, HEIGHT // 2))

```

User prompt: Please help me implement Task 4 for my Python balloon-popping game. Here are the exact requirements: The Goal: Implement a 30-second timed round and a restart mechanism. Requirements: Add a 30-second countdown timer that starts when the round begins. Display the remaining time clearly on the screen. The game should enter a "Game Over" state if EITHER the timer reaches zero OR the player's lives reach zero (from the previous task). During "Game Over": stop spawning balloons, stop updating balloon positions, and ignore clicks to pop balloons. On the "Game Over" screen, display the final score clearly. Provide a way for the player to start a new round (for example, pressing the 'R' key or clicking the screen). This must fully reset the score, lives, timer, and clear any remaining balloons on the screen. Please provide the updated game/game_engine.py code (and let me know if I need to update main.py to pass in keyboard events for the restart mechanism). Include a reset() method in the GameEngine class to cleanly handle restarting the round.

Response: Update game/game_engine.py to use Python's built-in time module. This ensures the 30-second countdown tracks accurately via the wall clock rather than relying on frame-rate approximations. The __init__ method is simplified to call a new reset() method, which handles establishing and restarting the round state.

game/game_engine.py

```

import random
import time

from game.balloon import Balloon
from game.click_detection import check_pop
from game.renderer import WIDTH, HEIGHT

SPAWN_INTERVAL_FRAMES = 45
ROUND_TIME_SECONDS = 30

class GameEngine:
    def __init__(self):
        self.reset()

```

```

def reset(self):
    """Resets the game state for a new round."""
    self.balloons = []
    self.frames_until_spawn = 0
    self.score = 0
    self.lives = 3
    self.game_over = False
    self.start_time = time.time()
    self.time_left = ROUND_TIME_SECONDS

def _spawn_balloon(self):
    radius = random.randint(16, 44)
    x = random.randint(radius + 10, WIDTH - radius - 10)
    speed = random.uniform(1.5, 3.0)
    self.balloons.append(Balloon(x=x, y=-radius, radius=radius, speed=speed))

def handle_click(self, pos):
    # Allow restarting the game via a mouse click if the game is over
    if self.game_over:
        self.reset()
        return

    popped = check_pop(self.balloons, pos)
    if popped is not None:
        self.balloons.remove(popped)
        self.score += popped.points
        if self.score < 0:
            self.score = 0

def update(self):
    if self.game_over:
        return

    # Update the countdown timer
    elapsed_time = time.time() - self.start_time
    self.time_left = max(0, int(ROUND_TIME_SECONDS - elapsed_time))

    # Check timeout condition
    if self.time_left <= 0:
        self.game_over = True
        return

    self.frames_until_spawn -= 1
    if self.frames_until_spawn <= 0:
        self._spawn_balloon()
        self.frames_until_spawn = SPAWN_INTERVAL_FRAMES

    for b in self.balloons:
        b.update()

    active_balloons = []
    for b in self.balloons:
        if b.is_past_bottom(HEIGHT):
            self.lives -= 1
        else:
            active_balloons.append(b)

    self.balloons = active_balloons

    # Check lives condition
    if self.lives <= 0:
        self.lives = 0
        self.game_over = True

def draw(self, surface, font):
    from game import renderer
    renderer.draw_scene(surface, self.balloons)

    renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))
    renderer.draw_text(surface, font, f"Lives: {self.lives}", (10, 40))
    renderer.draw_text(surface, font, f"Time: {self.time_left}s", (10, 70))

    if self.game_over:

```

```
# Adjust these coordinates depending on your display size and font
renderer.draw_text(surface, font, "GAME OVER", (WIDTH // 2 - 80, HEIGHT // 2 - 30))
renderer.draw_text(surface, font, f"Final Score: {self.score}", (WIDTH // 2 - 80, HEIGHT // 2 + 10))
renderer.draw_text(surface, font, "Click or Press 'R' to Restart", (WIDTH // 2 - 140, HEIGHT // 2 + 40))
```

To support the 'R' key restart requirement, you will need to intercept keyboard events in your main application loop. The mouse click restart is already handled natively in the `handle_click` logic above, but routing the 'R' key requires a small update to `main.py`.

main.py (Event Loop Snippet)

```
for event in pygame.event.get():
    if event.type == pygame.QUIT:
        running = False
    elif event.type == pygame.MOUSEBUTTONDOWN:
        if event.button == 1:
            engine.handle_click(event.pos)
    elif event.type == pygame.KEYDOWN:
        # Pass the restart trigger directly to the engine
        if event.key == pygame.K_r:
            engine.reset()
```
